Fixing `std::complex` binary operators

Document #:	P3788R0 [Latest] [Status]
Date:	2025-07-11
Project:	Programming Language C++
Audience:	SG6 Numerics Library Evolution Working Group

Reply-to:	Mateusz Pusz (Epam Systems) <mateusz.pusz@gmail.com>

Contents

§ 1 Introduction

This paper proposes a backward-compatible solution to fix the inconsistent behavior of `std::complex` binary operators with implicit conversions. The proposed changes:

1. Enable implicit conversions for scalar operands in `std::complex` binary operations.

2. Optionally support mixed-type complex operations.

3. Maintain full backward compatibility with existing code.

§ 2 Motivation

Consider the following natural mathematical operations with complex numbers:

```
long double ld = 2.5;
std::complex<float> c = 3.14;           // OK!!!
auto res1 = c * 3.14f;                  // OK
auto res2 = c * 3.14;                   // DOES NOT COMPILE!!!
auto res3 = c * ld;                     // DOES NOT COMPILE!!!
auto res4 = c * static_cast<float>(ld); // OK but verbose
if (c == 3.14f) {                       // OK
  // ...
}
if (c == 3.14) {                        // DOES NOT COMPILE!!!
  // ...
}
```

The above behavior is surprising and violates good design practices. It is a well-established design rule that if a type `T` is convertible to type `U`, then a binary operator on types `T` and `U` should also work correctly. This issue affects user experience and forces developers to explicitly cast literals and variables of compatible types, making mathematical code less readable and more error-prone.

Additionally, `std::complex` is problematic in generic contexts. Here is an excerpt from the mp-units library:

```
template<typename T>
using   scaling_factor_type_t   =   conditional<treat_as_floating_point<T>,
        long double, std::intmax_t>;

template<typename T>
concept ScalableByFactor = requires(const T v, const scaling_factor_type-
        _t<T> f) {
  { v* f } -> std::common_with<T>;
  { f* v } -> std::common_with<T>;
  { v / f } -> std::common_with<T>;
};

template<typename T>
concept  RealScalarRepresentation  =  (!Quantity<T>)  &&  RealScalar<T>  &&
        ScalableByFactor<T>;

template<typename T>
concept ComplexScalarRepresentation =
   (!Quantity<T>) && ComplexScalar<T> && requires(const T v, const scal-
        ing_factor_type_t<T> f) {
    // TODO The below conversion to `T` is an exception compared to other
        representation types
```

```
    // `std::complex<T>` * `U` do not work, but `std::complex<T>` is con-
        vertible from `U`
    { v* T(f) } -> std::common_with<T>;
    { T(f) * v } -> std::common_with<T>;
    { v / T(f) } -> std::common_with<T>;
  };

template<typename T>
concept    ScalarRepresentation    =    RealScalarRepresentation<T>    ||
        ComplexScalarRepresentation<T>;

template<typename T>
concept   VectorRepresentation   =   NotQuantity<T>   &&   Vector<T>   &&
        ScalableByFactor<T>;
```

As seen above, the general-purpose physical quantities and units library has to special-case the `std::complex` type to handle scaling of a quantity value between different units. This is wrong and should not be required for C++ Standard Library types.

§ 3 Problem statement

`std::complex` is implicitly constructible from its representation type `T`. The non-explicit constructor takes type `T` as a regular function parameter type (not a dependent name). This means that all standard and user-defined conversions apply to this function parameter in such a constructor call.

However, all binary operators are defined as non-member non-friend overloads where `T` is a deduced template parameter type. For example:

```
template<class T> constexpr complex<T> operator*(const complex<T>&, const
        complex<T>&);
template<class T> constexpr complex<T> operator*(const complex<T>&, const
        T&);
template<class   T>   constexpr   complex<T>   operator*(const   T&,   const
        complex<T>&);
```

According to template argument deduction rules, most conversions are not allowed for standalone function template parameters of type `T`. This means that there is no overload that can accept `std::complex<float>` and a `double` as its arguments.

Additionally, we cannot perform operations on complex numbers of different types (e.g., `complex<double>` and `complex<float>`).

§ 4 Hidden friends are not a solution here

Making those operators hidden friends would immediately solve all the issues described above. However, this would be an API break for any code that relies on them being declared at namespace scope, e.g., code that calls `std::operator*<float>(c, 3.14)`, or similar constructs.

§ 5 Fixing conversions to T scalar function template parameters

We propose to fix the conversions to `T` in a similar way that [P0558R1] fixed `std::atomic` interfaces. Instead of using type `T` as a function template parameter, we can use `typename std::complex<T>::value_type` instead:

```
template<class T> constexpr complex<T> operator*(const complex<T>&, const
        complex<T>&);
template<class T> constexpr complex<T> operator*(const complex<T>&, const
        typename complex<T>::value_type&);
template<class T> constexpr complex<T> operator*(const typename com-
        plex<T>::value_type&, const complex<T>&);
```

This change does not break any existing code and enables implicit conversions for `T` parameters.

§ 6 Fixing overloads with heterogeneous types

If we want to allow operators to work on `complex` class templates with different value types, we can add additional overloads to handle that:

```
template<typename T>
concept non-void = (!is_void_v<T>);

template<typename T, typename U>
  requires requires(T t, U u) { { t * u } -> non-void; }
auto operator*(complex<T>, complex<U>) -> complex<decltype(declval<T>() *
        declval<U>())>;
```

§ 7 Impact on existing code

The proposed solution is fully backward compatible:

- All existing code continues to compile and behave identically
- No performance impact on existing operations
- No changes to existing overload resolution for cases that currently compile
- The change only adds new viable overloads for previously ill-formed expressions

§ 8 Implementation considerations

Several standard library implementations have been consulted, and preliminary implementation shows that the proposed changes can be implemented without affecting existing optimizations or code generation.

§ 9 Wording

The proposed changes are relative to [N5008].

§ 9.1 [complex.syn]

Modify the declarations in [complex.syn] as follows:

```
template<class T> constexpr complex<T> operator+(const complex<T>& lhs,
        const complex<T>& rhs);
template<class T> constexpr complex<T> operator+(const complex<T>& lhs,
        const ~~typename complex::value_type~~& rhs);
template<class T> constexpr complex<T> operator+(const ~~typename com-~~
        ~~plex::value_type~~& lhs, const complex<T>& rhs);

template<class T> constexpr complex<T> operator-(const complex<T>& lhs,
        const complex<T>& rhs);
template<class T> constexpr complex<T> operator-(const complex<T>& lhs,
        const ~~typename complex::value_type~~& rhs);
template<class T> constexpr complex<T> operator-(const ~~typename com-~~
        ~~plex::value_type~~& lhs, const complex<T>& rhs);

template<class T> constexpr complex<T> operator*(const complex<T>& lhs,
        const complex<T>& rhs);
template<class T> constexpr complex<T> operator*(const complex<T>& lhs,
        const ~~typename complex::value_type~~& rhs);
template<class T> constexpr complex<T> operator*(const ~~typename com-~~
        ~~plex::value_type~~& lhs, const complex<T>& rhs);

template<class T> constexpr complex<T> operator/(const complex<T>& lhs,
        const complex<T>& rhs);
template<class T> constexpr complex<T> operator/(const complex<T>& lhs,
        const ~~typename complex::value_type~~& rhs);
template<class T> constexpr complex<T> operator/(const ~~typename com-~~
        ~~plex::value_type~~& lhs, const complex<T>& rhs);

template<class T> constexpr bool operator==(const complex<T>& lhs, const
        complex<T>& rhs);
template<class T> constexpr bool operator==(const complex<T>& lhs, const
        ~~typename complex::value_type~~& rhs);
```

§ 9.2 [complex.ops]

Modify the function specifications in [complex.ops] to match the declarations. The semantics remain unchanged, only the parameter types are modified to enable template argument deduction with implicit conversions.

For each affected function template, change the parameter type from `const T&` to `const typename complex<T>::value_type&` where applicable.

§ 9.3 Feature test macro

Add the following feature test macro to [version.syn]:

```
+#define   __cpp_lib_complex_implicit_conversions   ??????L   //   also   in
        <complex>
```

§ 9.4 Optional: Heterogeneous type support

If the committee decides to support operations between complex numbers of different types, add the following overloads:

```
template<typename T, typename U>
constexpr auto operator+(complex<T>, complex<U>) -> complex<decltype(de-
        clval<T>() + declval<U>())>;

template<typename T, typename U>
constexpr auto operator-(complex<T>, complex<U>) -> complex<decltype(de-
        clval<T>() - declval<U>())>;

template<typename T, typename U>
constexpr auto operator*(complex<T>, complex<U>) -> complex<decltype(de-
        clval<T>() * declval<U>())>;

template<typename T, typename U>
constexpr auto operator/(complex<T>, complex<U>) -> complex<decltype(de-
        clval<T>() / declval<U>())>;

template<typename T, equality_comparable_with<T> U>
constexpr bool operator==(complex<T>, complex<U>);
```

§ 10 Acknowledgements

Special thanks and recognition goes to Epam Systems for supporting Mateusz's membership in the ISO C++ Committee and the production of this proposal.

§ 11 References

[N5008] Thomas Köppe. 2025-03-15. Working Draft, Programming Languages — C++.
https://wg21.link/n5008

[P0558R1] Billy O'Neal. 2017-03-03. Resolving atomic<T> named base class inconsistencies.
https://wg21.link/p0558r1